```
        ...
‹else
  body ›
end
```

The second form of a case expression takes a target expression following the case keyword. It searches for a match starting at the first (top left) comparison, using the triple-equals operator: _comparison_ === _target_ (as a performance optimization, comparisons between literal strings and between numbers do not use ===):

```
case target
when ‹comparison›+ ‹then›
  body
when ‹comparison›+ ‹then›
  body
        ...
‹else
  body ›
end
```

A comparison can be an array reference preceded by an asterisk, in which case it is expanded into that array's elements before the tests are performed on each. When a comparison returns true, the search stops, and the body associated with the comparison is executed (nothing like a break statement is required). The case statement then returns the value of the last expression executed. If no comparison matches and an else clause is present, the body of that clause will be executed; otherwise, the case statement returns nil.

The then keyword separates the when comparisons from the bodies and is not needed if the body starts on a new line.

Case Pattern Matching

```
case expression
in pattern
  body
in pattern
  body
else
  body
end
```

Pattern matching is a powerful construct to extract values from complicated data structures.

The expression is an already existing piece of data, typically featuring nested hashes and arrays. The case statement selects the first branch where the pattern matches the expression, assigns any variables that are part of the pattern, executes the body associated with that branch and exits the case. If no patterns match, the else branch is executed. If no patterns match and there is no else branch, Ruby throws a NoMatchingPatternError.

Defining Patterns

The simplest pattern is just a Ruby object, the pattern matches an expression if the two values are ===. This is useful for classes that define === for extended matching, like Class and String, though there are also cases where you might just match against a regular literal value.

Even so, it's probably more useful matched with a data structure: